

# Control Statements and Program Development

## Lec-8

# Algorithms

- An algorithm is a procedure for solving a problem in terms of:
  - the actions to execute, and
  - the order in which these actions execute
- Ex : Algorithm to add 2 numbers
  - 1 : Take 2 numbers from the user as input
  - 2 : Store the result in a variable
  - 3 : Print or return the result

# Pseudocode

- Pseudocode is an informal English-like language for “thinking out” algorithms. You write text that describes what your program should do. You then convert the pseudocode to Python by replacing pseudocode statements with their Python equivalents.

*Prompt the user to enter the first integer*

*Input the first integer*

*Prompt the user to enter the second integer*

*Input the second integer*

*Add first integer and second integer, store their sum*

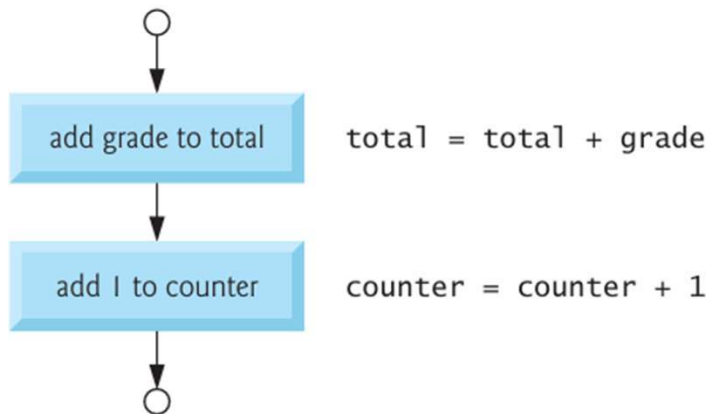
*Display the numbers and their sum*

# Forms of Control

- all programs could be written using three forms of control—namely, sequential execution, the selection statement and the repetition statement.

# Control Statements

- Usually, statements in a program execute in the order in which they're written. This is called sequential execution. Various Python statements enable you to specify that the next statement to execute may be other than the next one in sequence. This is called transfer of control and is achieved with Python control statements.



We use the rectangle (or action) symbol to indicate any action, such as a calculation or an input/output operation. The flowlines show the order in which the actions execute. First, the grade is added to the total, then 1 is added to the counter.

# Control Statements

- In a flowchart for a complete algorithm, the first symbol is a rounded rectangle containing the word “Begin.” The last symbol is a rounded rectangle containing the word “End.” In a flowchart for only a part of an algorithm, we omit the rounded rectangles, instead using small circles called connector symbols. The most important symbol is the decision (or diamond) symbol, which indicates that a decision is to be made, such as in an if statement.

# Selection Statements

- Python provides three types of selection statements that execute code based on a condition—an expression that evaluates to either True or False:
- The if statement performs an action if a condition is True or skips the action if the condition is False.
- The if...else statement performs an action if a condition is True or performs a different action if the condition is False.
- The if...elif...else statement performs one of many different actions, depending on the truth or falsity of several conditions.

# Selection Statement

- The if statement is called a single-selection statement because it selects or ignores a single action (or group of actions). The if...else statement is called a double-selection statement because it selects between two different actions (or groups of actions). The if...elif...else statement is called a multiple-selection statement because it selects one of many different actions (or groups of actions).

# Repetition Statements

- Python provides two repetition statements—while and for
- The while statement repeats an action (or a group of actions) as long as a condition remains True.
- The for statement repeats an action (or a group of actions) for every item in a sequence of items.

# Keywords

| Python keywords |        |        |          |       |        |       |
|-----------------|--------|--------|----------|-------|--------|-------|
| and             | as     | assert | async    | await | break  | class |
| continue        | def    | del    | elif     | else  | except | False |
| finally         | for    | from   | global   | if    | import | in    |
| is              | lambda | None   | nonlocal | not   | or     | pass  |
| raise           | return | True   | try      | while | with   | yield |

# If Statement

```
In [1]: grade = 85

In [2]: if grade >= 60:
...:     print('Passed')
...:
Passed
```

- If the condition is false, nothing is displayed, and the next pseudocode statement is “performed.” The pseudocode’s second line is indented. Python code requires indentation. Here it emphasizes that 'Passed' is displayed only if the condition is true.

# Suite Indentation

- Indenting a suite is required; otherwise, an `IndentationError` syntax error occurs:

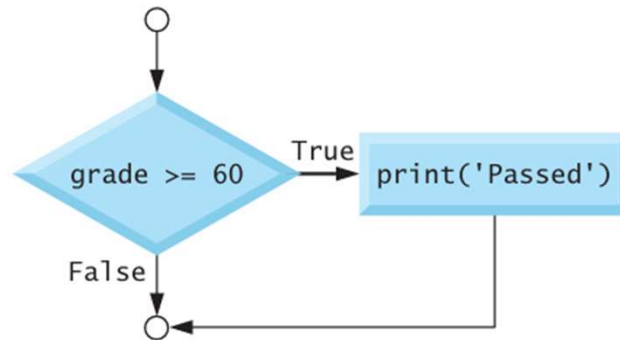
```
In [3]: if grade >= 60:
...:    print('Passed') # statement is not indented properly
File "<ipython-input-3-f42783904220>", line 2
    print('Passed') # statement is not indented properly
    ^
IndentationError: expected an indented block
```

```
In [4]: if grade >= 60:
...:     print('Passed') # indented 4 spaces
...:     print('Good job!') # incorrectly indented only two spaces
File <ipython-input-4-8c0d75c127bf>, line 3
    print('Good job!') # incorrectly indented only two spaces
    ^
IndentationError: unindent does not match any outer indentation level
```

# If statement flowchart

## if Statement Flowchart

The flowchart for the if statement in snippet [2] is:



The decision (diamond) symbol contains a condition that can be either **True** or **False**. The diamond has two flowlines emerging from it:

- One indicates the direction to follow when the condition in the symbol is **True**. This points to the action (or group of actions) that should execute.
- The other indicates the direction to follow when the condition is **False**. This skips the action (or group of actions).

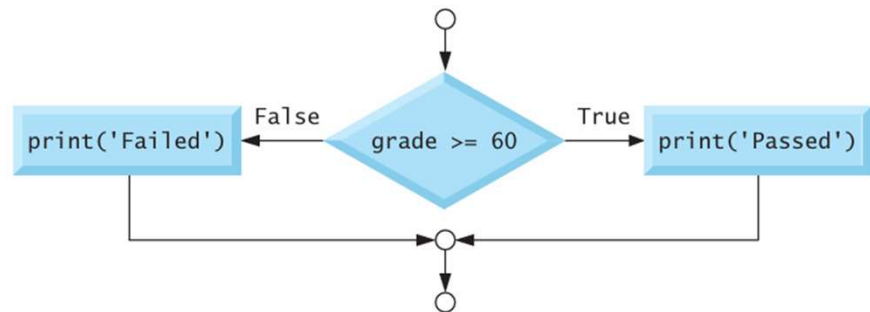
# if...else and if...elif...else Statements

```
In [1]: grade = 85

In [2]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Passed
```

## if...else Statement Flowchart

The flowchart below shows the preceding if...else statement's flow of control:



# if...else and if...elif...else Statements

The `else` in the `if...elif...else` statement is optional. Including it enables you to handle values that do not satisfy *any* of the conditions. When an `if...elif` statement without an `else` tests a value that does not make any of its conditions `True`, the program does not execute any of the statement's suites. The next statement in sequence after the `if...elif` statement executes. If you specify the `else`, you must place it after the last `elif`; otherwise, a `SyntaxError` occurs.

